

Université Ibn Tofail
Faculté des Sciences
Année universitaire : 2025-2026

MASTER INGENIERIE DU DEVELOPPEMENT D'APPLICATIONS ET DE SYSTEMES INTELLIGENTS

RAPPORT DE PROJET

Module DevOps

PLATEFORME DE GESTION DE TICKETS DE SUPPORT

Réalisé par :
LAGCIYER Mohammed-Reda
EL BAHY Mohammed

Encadrant :
Pr. Soufiane HAMIDA

TABLE DES MATIERES

1. Introduction	3
2. Architecture de l'application	4
3. Modèle de données	6
4. Fonctionnalités	7
5. Stratégie Git	8
6. Conteneurisation Docker	10
7. Pipeline CI/CD	12
8. Difficultés rencontrées et solutions	14
9. Répartition du travail	15
10. Conclusion et perspectives	16

1. Introduction

Dans le cadre du module DevOps, nous avons développé une plateforme de gestion de tickets de support (sujet n°05 : Mini Helpdesk). L'objectif de ce projet est de mettre en pratique les notions de versioning, de conteneurisation, et d'intégration continue apprises durant le module.

La plateforme permet à un utilisateur d'ouvrir un ticket de support avec une description et une priorité. Un agent peut ensuite consulter les tickets, y répondre via des commentaires, et faire évoluer leur statut (Ouvert, En cours, Résolu, Fermé). Un administrateur peut gérer les rôles des utilisateurs.

Pour mener à bien ce projet, nous avons utilisé les technologies suivantes : Symfony 7.4 pour le backend, React pour le frontend, PostgreSQL comme base de données, Docker pour la conteneurisation, et GitHub Actions pour le pipeline CI/CD.

Ce rapport présente les choix techniques que nous avons faits, la mise en œuvre de l'application, notre stratégie Git, notre pipeline CI/CD, ainsi que les difficultés que nous avons rencontrées et les solutions apportées.

2. Architecture de l'application

2.1 Vue globale

L'application est divisée en deux parties distinctes :

- Backend : API REST développée avec Symfony 7.4 (PHP 8.3)
- Frontend : Interface utilisateur développée avec React (Vite)

Ces deux parties communiquent via HTTP. Le frontend envoie des requêtes à l'API, qui retourne des données JSON. L'authentification est gérée par des tokens JWT (JSON Web Tokens).

2.2 Stack technique

Composant	Technologie
Backend	Symfony 7.4 / PHP 8.3
Frontend	React 19 / Vite / Tailwind CSS v4
Base de données	PostgreSQL 16
Serveur web	Nginx
Authentification	LexikJWT
State management frontend	Zustand
Conteneurisation	Docker / Docker Compose
CI/CD	GitHub Actions
Registre d'images	Docker Hub

2.1 Choix techniques

Nous avons choisi Symfony pour le backend car c'est un framework PHP mature, bien documenté, avec une gestion native de la sécurité et des migrations de base de données. React a été choisi pour le frontend car il permet de créer des interfaces dynamiques facilement. PostgreSQL est une base de données relationnelle robuste, adaptée à notre modèle de données. Docker permet de standardiser

l'environnement de développement entre les membres de l'équipe. GitHub Actions a été choisi pour le CI/CD car il s'intègre nativement avec notre dépôt GitHub.

3. Modèle de données

L'application repose sur trois entités principales :

User

Représente un utilisateur de la plateforme. Il possède un email, un mot de passe hashé, un prénom, un nom, et un ou plusieurs rôles (ROLE_USER, ROLE_AGENT, ROLE_ADMIN).

Ticket

Représente un ticket de support créé par un utilisateur. Il contient un titre, une description, une priorité (low, medium, high), un statut (open, in_progress, resolved, closed), et les dates de création et de modification. Un ticket appartient à un utilisateur.

Comment

Représente un commentaire ajouté sur un ticket. Il contient le contenu du commentaire et la date de création. Un commentaire est lié à un ticket et à un auteur (User).

Entité	Relation	Entité
User	OneToMany	Ticket
Ticket	OneToMany	Comment
User	OneToMany	Comment

4. Fonctionnalités

4.1 Authentification

- Inscription d'un nouvel utilisateur via `/api/auth/register`
- Connexion via `/api/auth/login` avec retour d'un token JWT
- Toutes les routes de l'API (sauf register et login) sont protégées par JWT
- Le token JWT est stocké côté frontend dans le `localStorage` via Zustand

4.2 Gestion des rôles

Trois rôles existent dans l'application :

- `ROLE_USER` : peut créer et consulter ses propres tickets, ajouter des commentaires.
- `ROLE_AGENT` : peut consulter tous les tickets, modifier les statuts, ajouter des commentaires.
- `ROLE_ADMIN` : peut gérer les rôles des utilisateurs via un dashboard dédié.

Le premier administrateur est créé via une commande Symfony :

```
php bin/console app:create-admin email password Prénom Nom
```

4.3 Gestion des tickets

- Création d'un ticket avec titre, description et priorité
- Liste des tickets filtrée selon le rôle (utilisateur voit ses tickets, agent voit tout)
- Consultation du détail d'un ticket
- Mise à jour du statut par un agent ou admin (Ouvert → En cours → Résolu → Fermé)
- Suppression d'un ticket par son auteur ou un admin

4.4 Commentaires

Tous les utilisateurs authentifiés peuvent ajouter des commentaires sur un ticket. Les commentaires sont affichés dans le détail du ticket avec le nom de l'auteur et la date de création. C'est le principal moyen de communication entre un utilisateur et un agent.

5. Stratégie Git

5.1 Workflow adopté

Nous avons adopté un workflow basé sur Git Flow simplifié avec trois niveaux de branches :

- main : branche de production, contient uniquement du code stable et testé
- develop : branche d'intégration, reçoit les fonctionnalités terminées via Pull Request
- feature/* : branches de fonctionnalités, créées depuis develop et mergées dans develop

5.2 Branches créées

Branch	Auteur	Description
feature/docker-setup	LAGCIYER Mohammed-Reda	Infrastructure Docker et configuration initiale
feature/migrations	EL BAHI Mohammed	Entités Doctrine et migrations
feature/auth	LAGCIYER Mohammed-Reda	Authentification JWT (backend + frontend)
feature/admin-setup	EL BAHI Mohammed	Commande de création d'admin
feature/admin-dashboard	EL BAHI Mohammed	Dashboard admin et gestion des rôles
feature/ticket-api	EL BAHI Mohammed	API CRUD tickets et commentaires
feature/ticket-ui	LAGCIYER Mohammed-Reda	Interface tickets et commentaires
feature/devops-setup	LAGCIYER Mohammed-Reda	Linter, Docker Hub, badge CI

5.3 Conventional Commits

Nous avons utilisé la convention **Conventional Commits** pour structurer nos messages de commit :

- feat: — nouvelle fonctionnalité
- fix: — correction de bug
- ci: — modification du pipeline CI/CD
- docs: — documentation
- chore: — tâches de maintenance

5.4 Pull Requests

Chaque fonctionnalité a été intégrée via une Pull Request sur GitHub. L'autre membre de l'équipe effectuait une revue de code avant le merge dans develop. Une fois plusieurs fonctionnalités validées dans develop, une PR develop → main était créée pour déployer en production.

6. Conteneurisation Docker

6.1 Structure

L'environnement Docker est composé de quatre services définis dans `docker-compose.yml` :

Service	Image	Rôle
st_postgres	postgres:16-alpine	Base de données PostgreSQL
st_php	Custom (PHP 8.3 FPM Alpine)	backend Symfony
st_nginx	nginx:alpine	Web server
st_frontend	node:20-alpine	Frontend React

6.2 Dockerfile PHP

Le Dockerfile du backend est basé sur `php:8.3-fpm-alpine`. Il installe les extensions PHP nécessaires (`pdo`, `pdo_pgsql`, `intl`, `zip`, `mbstring`, `opcache`), Composer 2.7, et la CLI Symfony. Un utilisateur non-root `appuser` est créé pour des raisons de sécurité, ce qui est une bonne pratique en production.

6.3 Nginx

Nginx joue le rôle de reverse proxy entre le frontend et le backend :

- Les requêtes vers `/api/` sont transmises au conteneur PHP via FastCGI (port 9000)
- Toutes les autres requêtes sont redirigées vers le serveur Vite (frontend, port 5173)

Ce système permet d'avoir un seul point d'entrée (port 80) pour l'ensemble de l'application.

6.4 Lancement

L'application se lance en une seule commande depuis la racine du projet :

`docker compose up --build -d`

Cette commande construit les images, crée les conteneurs, et les démarre en arrière-plan. Après le démarrage, il faut exécuter les migrations et créer le premier utilisateur admin manuellement (voir README).

7. Pipeline CI/CD

7.1 Déclencheurs

Le pipeline GitHub Actions se déclenche automatiquement dans deux cas :

- À chaque push sur les branches main ou develop
- À chaque Pull Request vers main ou develop

Cela garantit que tout code intégré a été validé par le pipeline avant le merge.

7.2 Jobs

Le pipeline est composé de trois jobs qui s'exécutent dans l'ordre suivant :

A. Backend Job:

- 1) Checkout du code source
- 2) Installation de PHP 8.3 avec les extensions nécessaires
- 3) Création du fichier .env depuis .env.example
- 4) Installation des dépendances via Composer
- 5) Analyse statique du code avec PHPStan (niveau 1)
- 6) Génération des clés JWT
- 7) Création et migration de la base de données de test
- 8) Exécution des tests automatisés avec PHPUnit

B. Frontend Job:

- 1) Checkout du code source
- 2) Installation de Node.js 24
- 3) Installation des dépendances npm
- 4) Analyse du code avec ESLint
- 5) Build de production avec Vite

Ce job s'exécute uniquement lors d'un push vers main, après la réussite des deux jobs précédents.

- 1) Connexion à Docker Hub avec les secrets GitHub
- 2) Build de l'image Docker du backend
- 3) Push de l'image vers ruddy99/support-ticket-devops:latest

7.3 Secrets Github

Les variables sensibles ne sont jamais stockées dans le code. Elles sont gérées via les secrets GitHub Actions :

Secret	Usage
POSTGRES_DB	Nom de la base de données
POSTGRES_USER	Utilisateur PostgreSQL
POSTGRES_PASSWORD	Mot de passe PostgreSQL
JWT_PASSPHRASE	Passphrase pour les clés JWT
DOCKERHUB_USERNAME	Nom d'utilisateur Docker Hub
DOCKERHUB_TOKEN	Token d'accès Docker Hub

8. Difficultés rencontrées et solutions

Interpolation des variables dans DATABASE_URL

Les variables d'environnement shell (\${POSTGRES_USER}) n'étaient pas résolues dans le conteneur PHP, ce qui causait des erreurs de connexion à la base de données. Solution : passer explicitement les variables via la section environment du service PHP dans docker-compose.yml.

Gestion du JWT entre développeurs

Chaque développeur doit générer ses propres clés JWT avec la même passphrase. Sans documentation claire, cela causait des erreurs 500 lors de la connexion. Solution : documenter la procédure dans le README et utiliser des secrets GitHub pour le pipeline.

Incompatibilité de version Node

Le package-lock.json généré localement avec Node 24 était incompatible avec Node 20 dans le pipeline CI, causant des échecs lors de npm ci. Solution : aligner la version Node du pipeline sur la version locale (Node 24) et utiliser npm install au lieu de npm ci.

Contexte de build Docker incorrect

Le Dockerfile référençait un fichier php.ini qui n'était pas présent dans le contexte de build ./backend. Solution : utiliser ./docker/php comme contexte de build dans le pipeline.

Conflit de merge avec main

Un merge accidentel d'une branche feature directement vers main a créé des conflits importants avec develop. Solution : revert de la PR sur GitHub, résolution des conflits en local, et recréation de la PR vers la bonne branche.

ESLint et fichiers shadcn/ui

Les composants auto-générés par shadcn/ui déclenchaient des erreurs ESLint car ils importaient React sans l'utiliser (React 17+ ne le requiert plus). Solution : exclure le dossier src/components/ui/ dans eslint.config.js.

9. Répartition du travail

Le travail a été réparti de manière équilibrée entre les deux membres de l'équipe. LAGCIYER Mohammed-Reda s'est principalement occupé de l'infrastructure DevOps et du frontend, tandis que EL BAH I Mohammed a pris en charge le développement du backend.

Tâche	LAGCIYER Mohammed-Reda	EL BAH I Mohammed
Infrastructure Docker	✓	
Pipeline CI/CD	✓	
Configuration Nginx	✓	
Entités Doctrine		✓
Migrations base de données		✓
API Authentification		✓
Frontend Authentification	✓	
Commande admin		✓
API Admin		✓
Dashboard Admin (frontend)		✓
API Tickets		✓
API Commentaires		✓
Interface Tickets (frontend)	✓	
Interface Commentaires (frontend)	✓	
Documentation README	✓	

10. Conclusion et perspectives

Ce projet nous a permis de mettre en pratique l'ensemble des notions DevOps vues en cours. Nous avons construit une application complète avec une chaîne CI/CD fonctionnelle, de la conteneurisation Docker, et un workflow Git professionnel.

Les principales compétences acquises sont : la mise en place d'un pipeline d'intégration continue avec GitHub Actions, la gestion sécurisée des secrets, et la conteneurisation d'une application full-stack avec Docker.

Le projet nous a également appris à gérer les problèmes concrets qui surviennent dans un environnement de développement réel : incompatibilités de versions, conflits de merge, erreurs de configuration Docker, et gestion des variables d'environnement entre plusieurs environnements.